THREE ADDRESS CODE (TAC) GENERATION

Aim

To write a C program that generates Three Address Code (TAC) for a given arithmetic expression by identifying operands and operators and using temporary variables according to operator precedence.

Algorithm

1. Start the program.

2. Read an arithmetic expression containing an assignment, such as a = b + c * d.

3. Separate the target variable from the right-hand-side expression.

4. Scan the right-hand-side expression from left to right.

5. Store operands in an operand stack and operators in an operator stack.

6. Apply operator precedence so that multiplication and division are processed before addition and subtraction.

7. For each processed operator, pop two operands, create a temporary variable, and print the TAC statement.

8. Push the temporary variable back onto the operand stack.

9. Process all remaining operators and assign the final result to the target variable.

10. Stop the program.

Pseudocode

```
START
Read expression
Separate target variable and RHS expression

FOR each symbol in RHS
    IF symbol is an operand
        Push operand onto operand stack
    ELSE IF symbol is '('
        Push it onto operator stack
    ELSE IF symbol is ')'
        Process operators until '(' is found
        Remove '('
    ELSE IF symbol is an operator
        WHILE top operator has higher or equal precedence
            Process top operator
        END WHILE
        Push current operator
    END IF
END FOR

Process all remaining operators
Pop final result
Print target = final result
STOP
```

Correct C Program

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
```

```
#define MAX 100

char operators[MAX];
char operands[MAX][MAX];
int opTop = -1, valTop = -1;
int tempCount = 1;

void pushOperator(char op) {
    operators[++opTop] = op;
}

char popOperator() {
    return operators[opTop--];
}

char peekOperator() {
    return operators[opTop];
}

void pushOperand(char value[]) {
    strcpy(operands[++valTop], value);
}

void popOperand(char value[]) {
    strcpy(value, operands[valTop--]);
}

int precedence(char op) {
    if (op == '+' || op == '-')
        return 1;
    if (op == '*' || op == '/')
        return 2;
    return 0;
}

void processOperator() {
    char op = popOperator();
    char left[MAX], right[MAX], temp[MAX];

    popOperand(right);
    popOperand(left);

    sprintf(temp, "t%d", tempCount++);

    printf("%s = %s %c %s\n", temp, left, op, right);

    pushOperand(temp);
}

void generateTAC(char expr[]) {
    int i = 0, k;
    char target[MAX] = "";
    char operand[MAX];

    /* Get target variable */
    k = 0;
    while (expr[i] != '=' && expr[i] != '\0') {
        if (!isspace((unsigned char)expr[i]))
            target[k++] = expr[i];
        i++;
    }
    target[k] = '\0';

    if (expr[i] == '=')
        i++;

    while (expr[i] != '\0') {

        if (isspace((unsigned char)expr[i])) {
```

```
            i++;
            continue;
        }

        if (isalnum((unsigned char)expr[i])) {
            k = 0;
            while (isalnum((unsigned char)expr[i]))
                operand[k++] = expr[i++];
            operand[k] = '\0';
            pushOperand(operand);
        }
        else if (expr[i] == '(') {
            pushOperator(expr[i++]);
        }
        else if (expr[i] == ')') {
            while (opTop >= 0 && peekOperator() != '(')
                processOperator();

            if (opTop >= 0)
                popOperator();

            i++;
        }
        else if (expr[i] == '+' || expr[i] == '-' ||
                 expr[i] == '*' || expr[i] == '/') {

            while (opTop >= 0 &&
                   peekOperator() != '(' &&
                   precedence(peekOperator()) >= precedence(expr[i]))
                processOperator();

            pushOperator(expr[i++]);
        }
        else {
            i++;
        }
    }

    while (opTop >= 0)
        processOperator();

    if (valTop >= 0)
        printf("%s = %s\n", target, operands[valTop--]);
}

int main() {
    char expr[MAX];

    printf("Enter an arithmetic expression: ");
    fgets(expr, MAX, stdin);

    printf("\nThree Address Code (TAC):\n");
    printf("-------------------------\n");

    generateTAC(expr);

    return 0;
}
```

Input

Enter an arithmetic expression in assignment form.

```
a = b + c * d
```

Output

```
Enter an arithmetic expression: a = b + c * d

Three Address Code (TAC):
-------------------------
t1 = c * d
t2 = b + t1
a = t2
```

Result

Thus, the C program was successfully implemented to generate Three Address Code (TAC) for an arithmetic expression. The program correctly follows operator precedence and uses temporary variables for intermediate results.